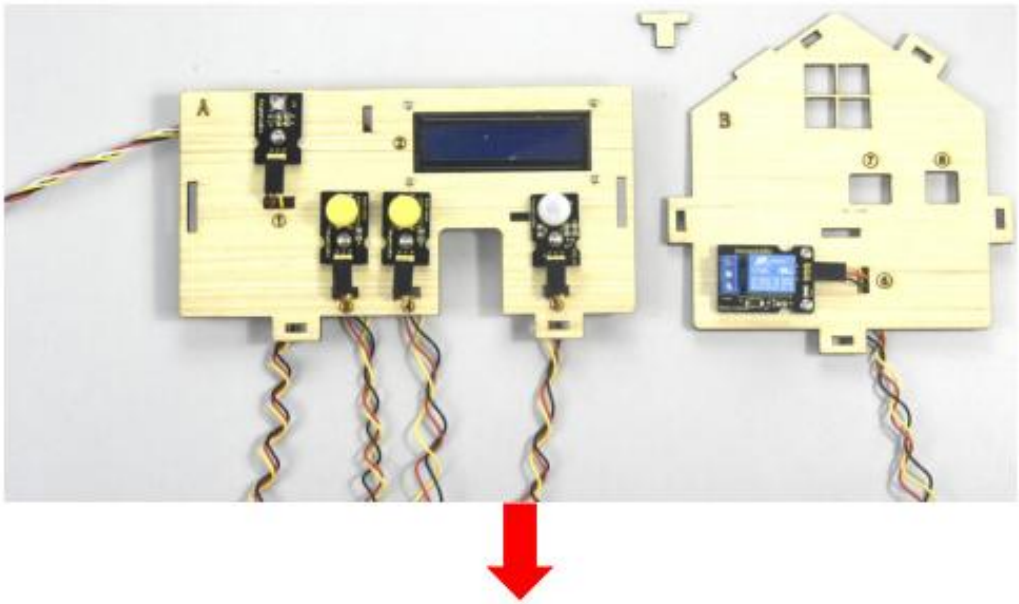
Οδηγός μαθημάτων Smart Home Kit για Arduino



Πώς δουλεύεις σε κάθε μάθημα

Ακολούθησε την ίδια σειρά κάθε φορά. Αν ολοκληρώσεις τα βήματα και περάσεις τον έλεγχο, μπορείς να προχωρήσεις στις προκλήσεις!

1. Κοίτα πρώτα την εικόνα του κυκλώματος και σημείωσε ποια εξαρτήματα και pins χρειάζεσαι.
2. Αποσύνδεσε το USB πριν αλλάξεις καλώδια. Σύνδεσε G, V και S όπως δείχνει η εικόνα και έλεγξε κάθε άκρο.
3. Σύνδεσε το USB, επίλεξε την πλακέτα Keyestudio / Arduino Uno **και τη σωστή θύρα**, έπειτα πάτησε Upload.
4. Κάνε την πρόκληση αλλάζοντας μία μόνο ιδέα κάθε φορά. Σημείωσε τι άλλαξες και τι παρατήρησες.

ΣΥΝΔΕΣΕΙΣ G = γείωση (GND) • V = τροφοδοσία (5V) • S = σήμα. Στον αισθητήρα ακολούθησε πάντα τα γράμματα του διαγράμματος. Μην αλλάζεις σύνδεση όσο η πλακέτα έχει ρεύμα.

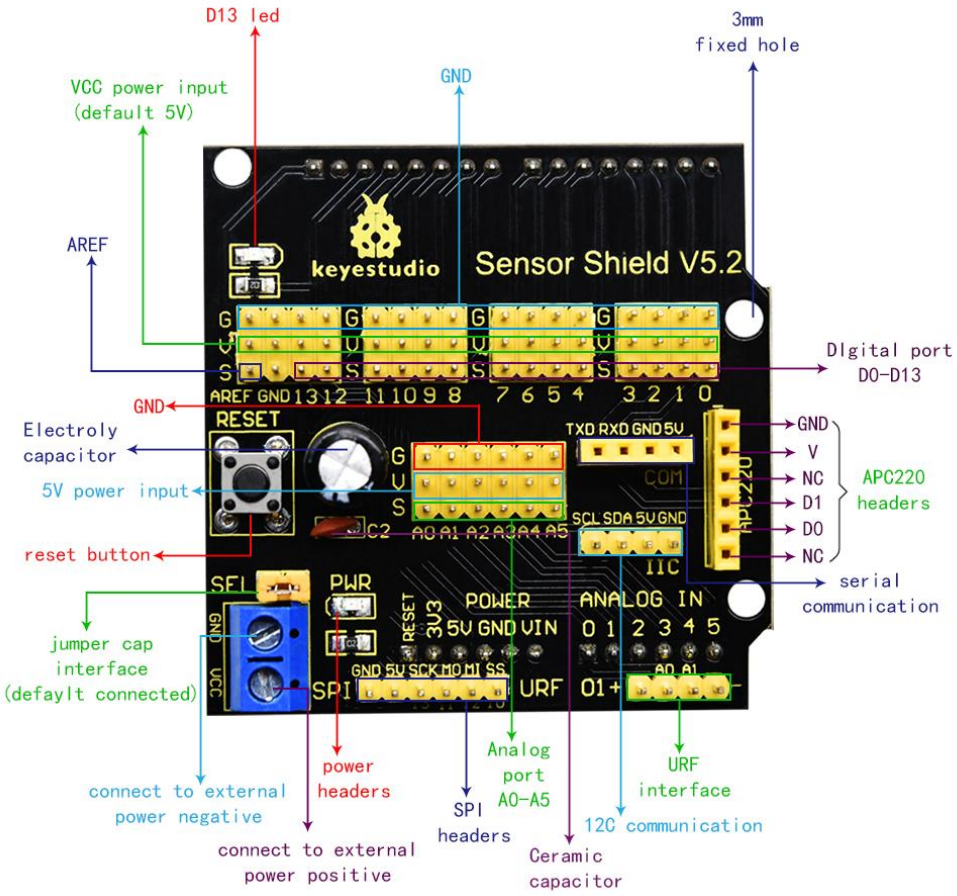
Γρήγορος έλεγχος πριν πατήσεις Upload

- Το USB ήταν αποσυνδεδεμένο όσο έφτιαχνα το κύκλωμα.
- Τα G, V και S ταιριάζουν με τις θέσεις του διαγράμματος.
- Η θύρα και η πλακέτα είναι επιλεγμένες σωστά.
- Έχω αποθηκεύσει αντίγραφο του αρχείου πριν πειράξω τον κώδικα.

Κανόνες ασφάλειας

- Το relay χρησιμοποιείται μόνο με το χαμηλής τάσης κύκλωμα του kit. Ποτέ σε πρίζα ή οικιακό ρεύμα.
- Ο MQ-2 είναι αισθητήρας για σχολικό πείραμα, όχι συναγερμός. Δεν τον δοκιμάζουμε με φλόγα, αναπτήρα ή οικιακό αέριο.
- Κρατάμε νερό και υγρασία μακριά από την πλακέτα. Ο αισθητήρας ατμού δεν είναι αδιάβροχος.
- Κρατάμε δάχτυλα, μαλλιά και χαλαρά αντικείμενα μακριά από τον ανεμιστήρα. Δεν πιέζουμε το servo με το χέρι.

Στον οδηγό του kit θα βρεις την πλήρη περιγραφή κάθε Project. Οι αριθμοί σελίδων που αναφέρονται στα φύλλα αντιστοιχούν στο PDF KS0085.

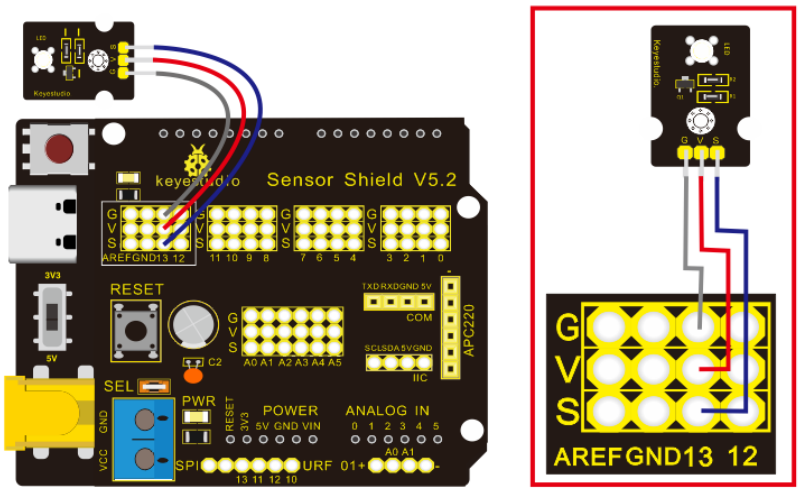

D13 led
3mm
fixed hole
GND
VCC power input
(default 5V)
AREF
keyestudio
Sensor Shield V5.2
Dlgital port
D0-D13
GND
Electroly
capacitor
5V power input
reset button
RESET
TXD RXD GND 5V
COM
SCL SDA 5V GND
IIC
APC220
GND
V
NC
D1
D0
NC
APC220
headers
serial
communication
SEL
PWR
POWER
ANALOG IN
3V3 5V GND VIN
0 1 2 3 4 5
jumper cap
interface
(defaylt connected)
GND 5V SCK MO MI SS
SPI
URF
connect to external
power negative
power
headers
connect to external
power positive
SPI
headers
Analog
port
A0-A5
Ceramic
capacitor
12C communication
URF
interface

Μάθημα 01 - LED Blink

Στόχος Σύνδεσε το LED module στα G, V και D13.

Συνδεσμολογία

Σύνδεσε το LED module στα G, V και D13.



Διάγραμμα σύνδεσης του λευκού LED στην έξοδο D13.
Στα καλώδια 3 pins: G = γείωση • V = 5V • S = σήμα

Προγραμματισμός

Για τον έλεγχο του LED θα χρησιμοποιησουμε την συνάρτηση digitalWrite
Η συνάρτηση **digitalWrite()** χρησιμοποιείται στον προγραμματισμό του Arduino για να στείλει ψηφιακό σήμα σε ένα pin (ακροδέκτη), θέτοντας την τάση του είτε σε **HIGH** (υψηλή τάση, συνήθως 5V ή 3.3V) είτε σε **LOW** (μηδενική τάση, 0V/γείωση)

```
digitalWrite(pin, value);
```

- **pin:** Ο αριθμός του ψηφιακού ακροδέκτη που θέλετε να ελέγξετε (π.χ. 13, A0).
- **value**: Η τιμή που θέλετε να δώσετε. Παίρνει μόνο δύο τιμές: **HIGH** (ενεργοποίηση/ρεύμα) ή **LOW** (απενεργοποίηση/διακοπή).

Βασικοί Κανόνες & Λειτουργία

- **Απαραίτητη προϋπόθεση (pinMode)**: Πριν χρησιμοποιήσετε την digitalWrite(), πρέπει οπωσδήποτε να δηλώσετε στο setup() ότι το συγκεκριμένο pin λειτουργεί ως έξοδος, χρησιμοποιώντας την εντολή pinMode(pin, OUTPUT).
- **Τι κάνει το HIGH**: Στέλνει τη μέγιστη τάση της πλακέτας (5V στα περισσότερα Arduino όπως το Uno, ή 3.3V σε νεότερα/ESP32) για να ανάψει ένα LED ή να ενεργοποιήσει ένα ρελέ.
- **Τι κάνει το LOW**: Μηδενίζει την τάση (0V), σβήνοντας την συνδεδεμένη συσκευή.

Παράδειγμα προγράμματος

```
const int LED_PIN = 13;
void setup() {
  pinMode(LED_PIN, OUTPUT);
}
void loop() {
  digitalWrite(LED_PIN, HIGH);
  delay(1000);
  digitalWrite(LED_PIN, LOW);
  delay(1000);
}
```

Έλεγχος

Πάτα upload και βεβαίωσου ότι το LED αναβοσβήνει όπως πρέπει κάθε 1s. Αν δεν λειτουργεί έλεγξε:

- Ότι ο κώδικας έγινε verify and upload success
- Θύρες COM/ttyUSB είναι σωστές
- Ότι οι συνδέσεις είναι σωστές

Αποστολές

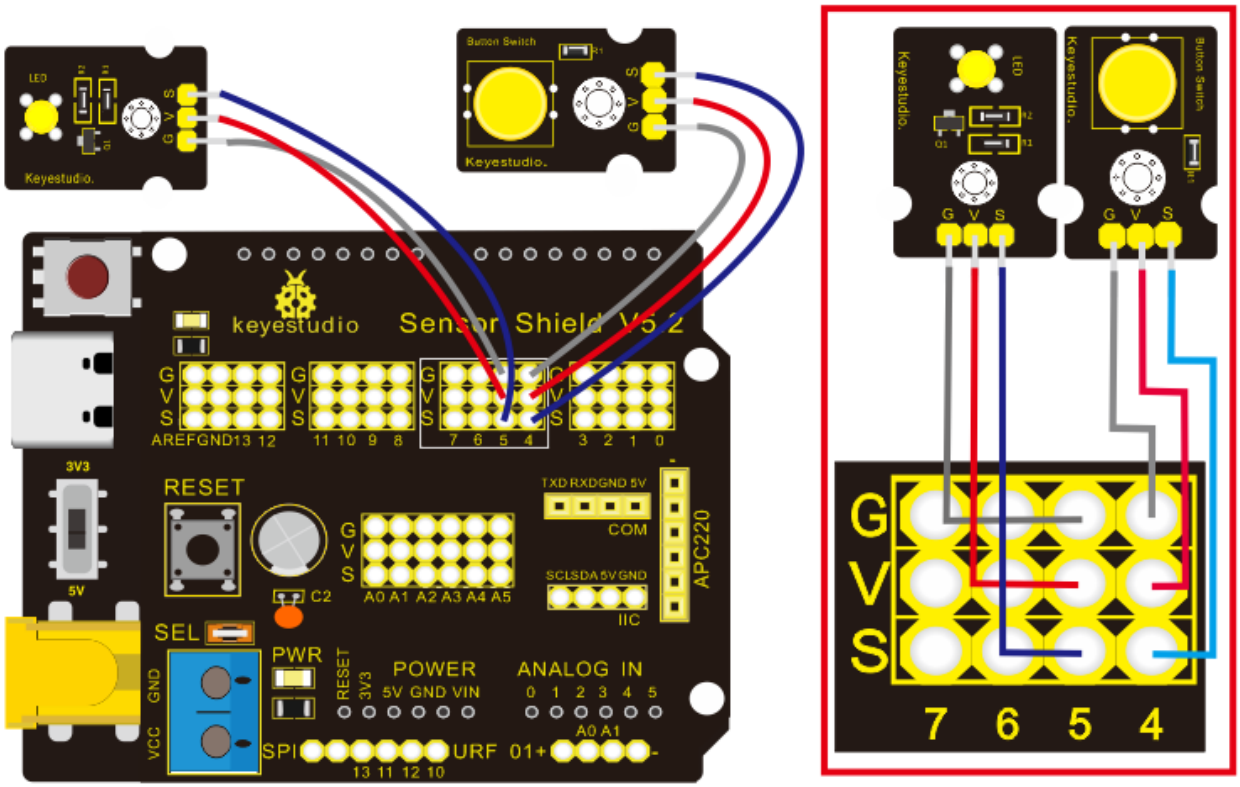
1. Αλλάξτε τον ρυθμό που αναβοσβήνει το LED έτσι ώστε να μην γίνεται αντιληπτό από τα μάτια μας
2. Προσθέστε ένα 2ο LED. Όταν το 1ο αναβεί το 2ο πρέπει να σβήνει
3. Super challenge! Φτιάξε μια αλληλουχία blinks με 2 LED. Οι συμμαθητές σας θα πρέπει να την δουν και να την αντιγράψουν! (Το ίδιο και εσείς..)

Μάθημα 02 - Button



Συνδεσμολογία

Σύνδεσε το button module στα G, V και D4 και το LED στο D5 όπως δείχνει η εικόνα.



Προγραμματισμός

Για τον έλεγχο της κατάστασης του κουμπιού θα χρησιμοποιήσουμε την «αντίθετη» συνάρτηση από την digitralWrite, την digitalRead. Η συνάρτηση **digitalRead()** χρησιμοποιείται για να διαβάσει την κατάσταση ενός ψηφιακού ακροδέκτη (pin), δηλαδή αν σε αυτόν εφαρμόζεται τάση (**HIGH**) ή όχι (**LOW**).

```
buttonState = digitalRead(pin);
```

- **pin (Παράμετρος εισόδου):** Ο αριθμός του ψηφιακού ακροδέκτη που θέλετε να ελέγξετε. Στο δικό σας παράδειγμα, εδώ βάζουμε το **4** (ή τη μεταβλητή buttonPin).
- **Επιστροφή (Τιμή που επιστρέφει):** Η συνάρτηση επιστρέφει μία από δύο πιθανές καταστάσεις, ανάλογα με την τάση που ανιχνεύει στο pin εκείνη ακριβώς τη στιγμή:
    - **HIGH**: Αν στο pin υπάρχει υψηλή τάση (5V ή 3.3V). Αυτό συνήθως σημαίνει ότι το κουμπί είναι πατημένο.
    - **LOW**: Αν στο pin δεν υπάρχει τάση (0V / γείωση). Αυτό σημαίνει ότι το κουμπί είναι ελεύθερο.

Βασικοί Κανόνες & Λειτουργία

- **Απαραίτητη προϋπόθεση (pinMode)**: Πριν διαβάσετε το pin, πρέπει να το δηλώσετε ως είσοδο στο setup() χρησιμοποιώντας την εντολή pinMode(4, INPUT);.
- **Αποθήκευση τιμής**: Επειδή η digitalRead() επιστρέφει μια τιμή (HIGH ή LOW), τη βάζουμε πάντα μέσα σε μια μεταβλητή (π.χ. buttonState), ώστε μετά να την ελέγξουμε με μια if.

Παράδειγμα προγράμματος 1

```
const int buttonPin = 4; // Το κουμπί στο Pin 4
int buttonState = 0;    // Μεταβλητή για να αποθηκεύουμε το HIGH ή LOW

void setup() {
  pinMode(buttonPin, INPUT);  // Ορισμός του Pin 4 ως Είσοδο (διαβάζει)
  Serial.begin(9600);       // Ενεργοποίηση της επικοινωνίας με τον υπολογιστή
}

void loop() {
  // Διαβάζει το Pin 4 και αποθηκεύει το αποτέλεσμα (HIGH ή LOW)
  buttonState = digitalRead(buttonPin);

  // Εμφανίζει την τιμή στην οθόνη του υπολογιστή
  Serial.print("Button State: ");
  Serial.println(buttonState);

  delay(100); // Μικρή καθυστέρηση για να προλαβαίνουμε να διαβάζουμε
}
```

Έλεγχος:

1. Πάτα **Upload** στο Arduino IDE.
2. Βεβαιώσου ότι οι θύρες **COM/ttyUSB** και η πλακέτα είναι σωστά επιλεγμένες.
3. Άνοιξε το **Serial Monitor** (το εικονίδιο με τον μεγεθυντικό φακό πάνω δεξιά στο Arduino IDE).
4. **Τι πρέπει να βλέπεις:** Όταν δεν πατάς το κουμπί πρέπει να γράφει 0 (LOW). Όταν το πατάς και το κρατάς, πρέπει να αλλάζει αμέσως σε 1 (HIGH).

Τώρα που ξέρουμε ότι το κουμπί μας δουλεύει **τέλεια** 😊, συνδέουμε και το LED στο **Pin 5** για να επηρεάζει η είσοδος (κουμπί) την έξοδο (LED).

Παράδειγμα προγράμματος 2

```
const int buttonPin = 4; // Το κουμπί στο Pin 4
const int ledPin = 5;   // Το LED στο Pin 5

int buttonState = 0;    // Μεταβλητή για να αποθηκεύουμε το HIGH ή LOW

void setup() {
 pinMode(buttonPin, INPUT);  // Ορισμός του Pin 4 ως Είσοδο (διαβάζει)
 pinMode(ledPin, OUTPUT);    // Ορισμός του Pin 5 ως Έξοδο (γράφει)
}

void loop() {
 // 1. Διαβάζει το Pin 4 και αποθηκεύει το αποτέλεσμα (HIGH ή LOW)
 buttonState = digitalRead(buttonPin);

 // 2. Ελέγχει τι τιμή πήρε η digitalRead
 if (buttonState == HIGH) {
  digitalWrite(ledPin, HIGH); // Αν διαβάσει HIGH, ανάβει το LED
 } else {
  digitalWrite(ledPin, LOW);  // Αν διαβάσει LOW, σβήνει το LED
 }
}
```

Έλεγχος

Πάτα upload και βεβαίωσου ότι το LED ανάβει όταν πατάμε το κουμπι και σβήνει όταν το αφήνουμε:

- Ότι ο κώδικας έγινε verify and upload success
- Θύρες COM/ttyUSB είναι σωστές
- Ότι οι συνδέσεις είναι σωστές

Αποστολές

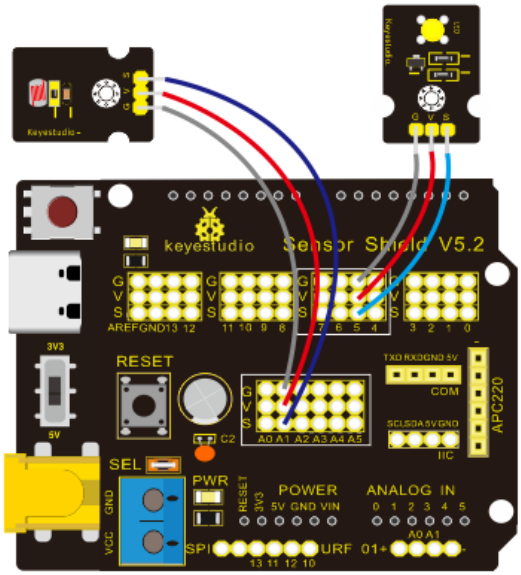
1. Κάντε το LED να λειτουργεί ανάποδα! Όταν **δεν** πατάτε το κουμπί, το LED πρέπει να είναι αναμμένο. Όταν **πατάτε** το κουμπί, το LED πρέπει να σβήνει.
2. Όσο κρατάτε πατημένο το κουμπί, το LED δεν πρέπει απλά να μένει αναμμένο, αλλά να **αναβοσβήνει γρήγορα** (σαν φλας αυτοκινήτου).
3. Μέχρι τώρα, το LED σβήνει μόλις αφήσετε το κουμπί. Τώρα θέλουμε το κουμπί να λειτουργεί σαν διακόπτης δωματίου: Πατάτε και αφήνετε μια φορά το LED ανάβει και μένει αναμμένο. Πατάτε και αφήνετε δεύτερη φορά το LED σβήνει. (Hint: Θα χρειαστείτε και ένα μικρό delay(200) για να μην μπερδεύεται το Arduino από το bounce του κουμπιού).
4. ****Όταν ξεκινάει το πρόγραμμα, το LED είναι σβηστό. Το Arduino περιμένει έναν τυχαίο χρόνο (π.χ. από 2 έως 5 δευτερόλεπτα). Ξαφνικά, το LED ανάβει!**
   Εσείς πρέπει να πατήσετε το κουμπί όσο πιο γρήγορα μπορείτε. Αν προλάβετε να το πατήσετε μέσα σε 300 milliseconds, το LED αναβοσβήνει 3 φορές χαρούμενα (Νίκη!). Αν αργήσετε, το LED σβήνει αργά (Έχασες!).**

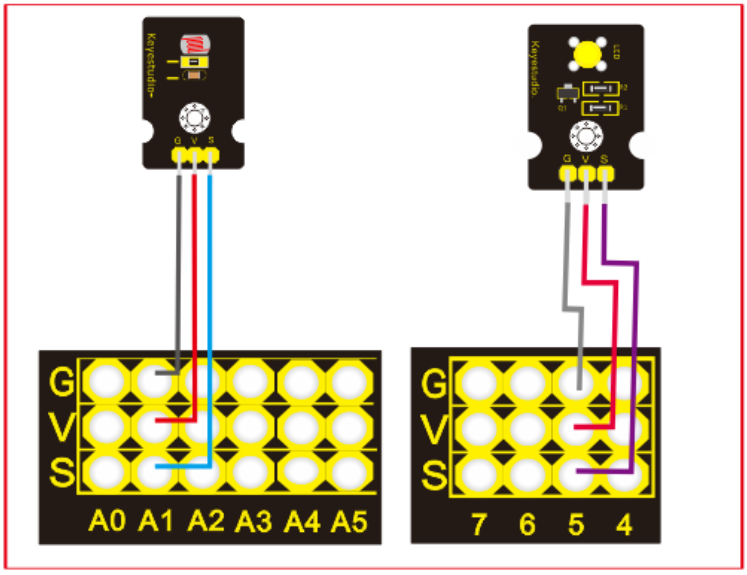
Μάθημα 03 - Μετρώ το φως



Συνδεσμολογία

Σύνδεσε το LDR (Light Dependent Resistance) module στα G, V και A0 και το LED στο D5 όπως δείχνει η εικόνα. Το LDR είναι μια αντίσταση η τιμή της οποίας αλλάζει με το φως! Μπορείτε με ένα πολύμετρο να μετρήσετε και να δειτε ακριβώς πως αλλάζει ανάλογα τον φωτισμό!





Προγραμματισμός

Για τον έλεγχο της έντασης του φωτός θα χρησιμοποιήσουμε μια νέα συνάρτηση, την analogRead.

Η συνάρτηση analogRead() χρησιμοποιείται για να διαβάσει την κατάσταση ενός αναλογικού ακροδέκτη (pin). Μετατρέπει την τάση που στέλνει το LDR module σε έναν ψηφιακό αριθμό από το 0 έως το 1023.

```
ldrValue = analogRead(pin);
```

- **pin (Παράμετρος εισόδου):** Ο αριθμός του αναλογικού ακροδέκτη που θέλετε να ελέγξετε. Στο δικό μας παράδειγμα, εδώ βάζουμε το **A0** (ή τη μεταβλητή ldrPin).
- **Επιστροφή (Τιμή που επιστρέφει):** Η συνάρτηση επιστρέφει έναν ακέραιο αριθμό από 0 έως 1023, ανάλογα με το φως που πέφτει στο module εκείνη την ακριβώς τη στιγμή:
    - **Μεγάλος αριθμός (κοντά στο 1023):** Αντιστοιχεί σε υψηλή τάση (5V). Αυτό σημαίνει ότι στο δωμάτιο υπάρχει **πολύ φως**.
    - **Μικρός αριθμός (κοντά στο 0):** Αντιστοιχεί σε χαμηλή τάση (0V). Αυτό σημαίνει ότι υπάρχει **σκοτάδι**.

Βασικοί Κανόνες & Λειτουργία

- **Αναλογικές Θύρες:** Οι αναλογικοί ακροδέκτες (A0-A5) λειτουργούν αυτόματα ως είσοδοι, επομένως δεν είναι απαραίτητο να γράψουμε pinMode(A0, INPUT); στο setup().
- **Αποθήκευση τιμής:** Επειδή η analogRead() επιστρέφει μια μεταβαλλόμενη τιμή (0-1023), τη βάζουμε πάντα μέσα σε μια μεταβλητή (π.χ. ldrValue), ώστε μετά να την εμφανίσουμε ή να την ελέγξουμε με μια if.

Παράδειγμα Προγράμματος 1

```
const int ldrPin = A0;  // Το μεσαίο σημείο του διαιρέτη τάσης στην Αναλογική θύρα A0
int ldrValue = 0;      // Μεταβλητή τύπου int για να αποθηκεύουμε την ένταση του φωτός

void setup() {
 Serial.begin(9600);   // Ενεργοποίηση της επικοινωνίας με τον υπολογιστή
}

void loop() {
 // Διαβάζει την αναλογική τιμή από το breadboard (από 0 έως 1023)
 ldrValue = analogRead(ldrPin);

 // Εμφανίζει την τιμή στην οθόνη του υπολογιστή
 Serial.print("LDR Value: ");
 Serial.println(ldrValue);

 delay(100); // Μικρή καθυστέρηση για να προλαβαίνουμε να διαβάζουμε
}
```

Έλεγχος:

1. Πάτα **Upload** στο Arduino IDE.
2. Βεβαιώσου ότι οι θύρες COM/ttyUSB και η πλακέτα είναι σωστά επιλεγμένες.
3. Άνοιξε το **Serial Monitor** (το εικονίδιο με τον μεγεθυντικό φακό πάνω δεξιά στο Arduino IDE).
4. **Τι πρέπει να βλέπεις:** Όταν ρίχνεις φως, ο αριθμός πρέπει να **ανεβαίνει** προς το 1023. Όταν το κλείνεις με το χέρι σου (σκοτάδι), ο αριθμός πρέπει να **πέφτει** προς το 0.
5. **Κράτα σημείωση:** Δες τι τιμή γράφει όταν το σκεπάζεις τελείως (π.χ. 280). Αυτό θα είναι το "όριο σκοταδιού" σου.

Παράδειγμα προγράμματος 2

```
const int ldrPin = A0;  // Το LDR στο Pin A0
const int ledPin = 5;   // Το LED στο Pin 5
int ldrValue = 0;      // Μεταβλητή int για να αποθηκεύουμε το φως

void setup() {
 pinMode(ledPin, OUTPUT);    // Ορισμός του Pin 5 ως Έξοδο (γράφει)
}
void loop() {
 // 1. Διαβάζει το Pin A0 και αποθηκεύει το αποτέλεσμα (0 έως 1023)
 ldrValue = analogRead(ldrPin);

 // 2. Ελέγχει αν η τιμή έπεσε κάτω από 300 (δηλαδή αν σκοτείνιασε)
 if (ldrValue < 300) {
  digitalWrite(ledPin, HIGH); // Αν σκοτείνιασε, άναψε το LED
 } else {
  digitalWrite(ledPin, LOW);  // Αν έχει φως, σβήσε το LED
 }
}
```

Έλεγχος

Πάτα **Upload** και βεβαιώσου ότι το LED ανάβει αυτόματα όταν κλείνεις το LDR με το χέρι σου και σβήνει όταν το αφήνεις:

1. Ότι ο κώδικας έγινε verify and upload success
2. Θύρες COM/ttyUSB είναι σωστές
3. Ότι οι συνδέσεις είναι σωστές
4. Ανάλογα τις συνθήκες φωτισμού του δωματίου ίσως χρειάζεται να αλλάξετε το όριο σκοταδιού!

Αποστολές!

1. Φτιάξτε ένα πρόγραμμα που θα αυξάνει την τιμή μιας μεταβλητής τύπου int που θα λέγεται dark όση ώρα έχουμε σκοτάδι.
   Θα χρειαστεί να δημιουργήσετε μια μεταβλητή στην αρχή του προγράμματος, πριν την void setup().
   Στη συνέχεια μέσα στη void loop ελέγχουμε αν έπεσε η φωτεινότητα (πχ analogRead(A0)<300). Τότε θα αυξήσουμε κατά 1 την τιμή της μεταβλητής (dark = dark +1 ή dark++).
2. Στο Παράδειγμα , το LED ανάβει απότομα. Τώρα θέλουμε να ανάβει μόνο αν το σκοτάδι κρατήσει για **αρκετή ώρα** (ώστε να μην ανάβει άσκοπα αν απλά περάσει μια στιγμιαία σκιά ή ένα πουλί μπροστά από τον αισθητήρα).
3. Φτιάξτε έναν συναγερμό για το ψυγείο! Όταν το LDR είναι στο σκοτάδι (κλειστή πόρτα), το LED είναι σβηστό. Αν κάποιος ανοίξει την πόρτα και μπει φως, το Arduino πρέπει να περιμένει 5 δευτερόλεπτα (χρόνος για να πάρεις κάτι). Αν η πόρτα μείνει ανοιχτή παραπάνω, το LED πρέπει να **αναβοσβήνει τρελά** για να ειδοποιήσει ότι ξεχάστηκε ανοιχτή! Στο επόμενο μάθημα που θα βάλουμε ηχείο αυτό θα είναι πολύ πιο ενδιαφέρον!

Οι εντολές if και while

Η **if** χρησιμοποιείται για να εκτελέσει μια εντολή μόνο αν ισχύει μια συνθήκη. Μπορούμε, για παράδειγμα, να ανάβουμε ένα LED όταν πατάμε το κουμπί:

```
if (digitalRead(koumpi) == LOW) {
  digitalWrite(led, HIGH);  // Ανάβει όταν πατηθεί το κουμπί
} else {
  digitalWrite(led, LOW);   // Σβήνει όταν αφεθεί
}
```

Η εντολή while **επαναλαμβάνει** τις εντολές που βρίσκονται μέσα στις αγκύλες { }, όσο μια συνθήκη είναι αληθής.

```
if (digitalRead(κουμπί) == HIGH) {
  digitalWrite(led, HIGH);
  // Αν πατηθεί το κουμπί, ανάβει το LED
}

while (digitalRead(κουμπί) == HIGH) {
  // Περιμένει όσο το κουμπί παραμένει πατημένο
}

digitalWrite(led, LOW);  // Σβήνει το LED όταν αφεθεί το κουμπί
```

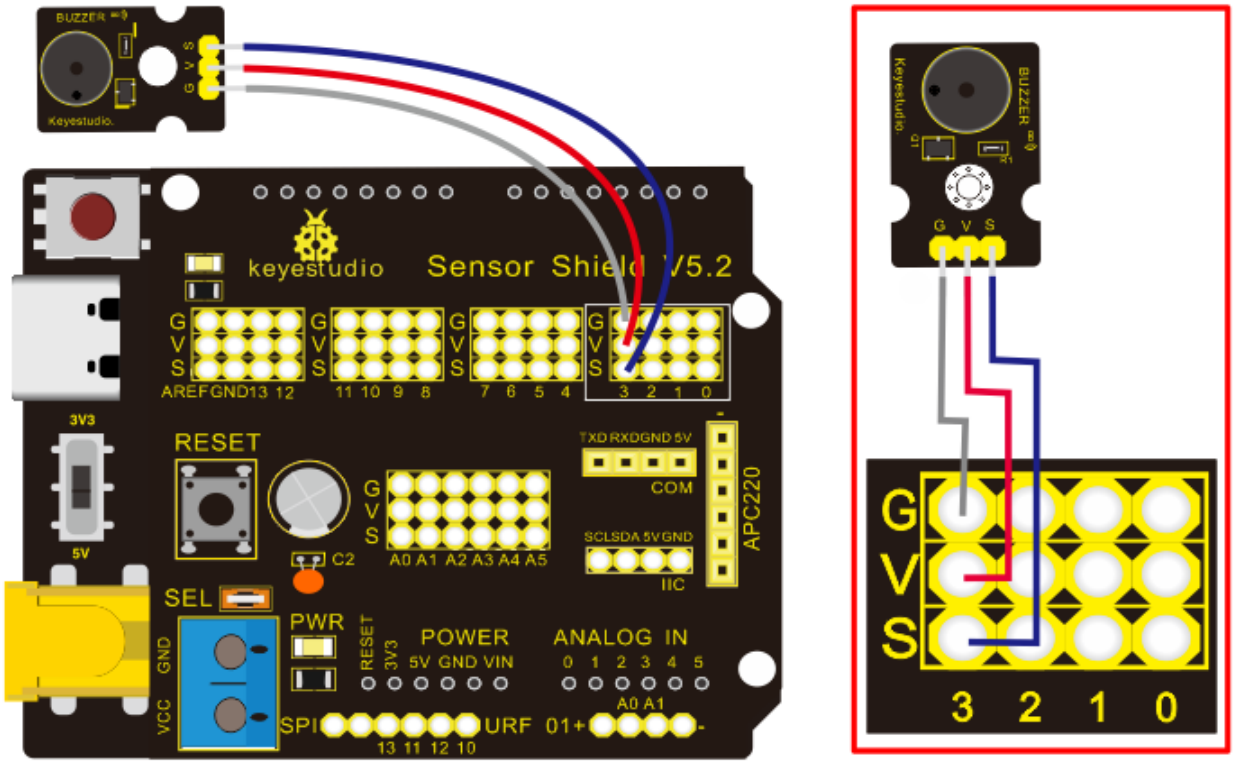
Μάθημα 04 - Ηχος!



Συνδεσμολογία

Σύνδεσε τα pins του **Passive Buzzer module** απευθείας στην πλακέτα με τα καλώδια (Jumper wires):

- Το pin **G** (GND / Γείωση) συνδέεται στο **G** (GND) του Arduino.
- Το pin **V** (VCC / Τροφοδοσία) συνδέεται στο **V** (5V) του Arduino.
- Το pin **S** (Signal / Σήμα) συνδέεται στο ψηφιακό **Pin 3** του Arduino.



Το **Passive Buzzer Module** περιλαμβάνει το εξάρτημα ενσωματωμένο πάνω σε μια μικρή πλακέτα.
Αντίθετα με ένα ενεργό (active) buzzer, το παθητικό buzzer λειτουργεί σαν ένα **μικροσκοπικό ηχείο**.

- Αν του δώσουμε απλά σταθερό ρεύμα (HIGH), δεν θα βγάλει κανέναν ήχο (ίσως μόνο ένα αχνό "κλικ").
- Για να βγάλει ήχο, πρέπει να του στείλουμε ένα σήμα που αναβοσβήνει με συγκεκριμένη συχνότητα (παλμούς), αναγκάζοντας την εσωτερική του μεμβράνη να δονηθεί.
- Το μεγάλο του πλεονέκτημα είναι ότι αλλάζοντας τη συχνότητα, αλλάζουμε τη νότα! Έτσι, **μπορεί να παίξει διαφορετικές μουσικές νότες και μελωδίες**.

Για να καταλάβουμε πώς το Arduino παράγει ήχους στο Buzzer ή πώς αναβοσβήνει ένα LED τόσο γρήγορα που "ξεγελάει" το μάτι, πρέπει να γνωρίζουμε δύο βασικές έννοιες: την **Περίοδος** και τη **Συχνότητα**.

- **Περίοδος (T):** Είναι ο **συνολικός χρόνος** που χρειάζεται για να ολοκληρωθεί **ένας πλήρης κύκλος** ενός φαινομένου που επαναλαμβάνεται (π.χ. το LED ανάβει ΚΑΙ σβήνει). Μετριέται σε δευτερόλεπτα (s).
- **Συχνότητα (f):** Είναι το **πόσες φορές** επαναλαμβάνεται αυτός ο κύκλος μέσα σε **1 δευτερόλεπτο**. Μετριέται σε **Hertz (Hz)**

Η μαθηματική τους σχέση είναι η εξής:

$$συχνότητα = \frac{1}{περίοδος} \rightarrow f = \frac{1}{T}$$

Στον βασικό κώδικα όπου το LED αναβοσβήνει ανά 1 δευτερόλεπτο:

```
digitalWrite(ledPin, HIGH); delay(1000);
digitalWrite(ledPin, LOW);  delay(1000);
```

Ερώτηση
Πόσο λέτε ότι είναι η περίοδος;
Πόσο είναι η συχνότητα;

T= .... s f=.... Hz

Το βιολογικό όριο και η ανθρώπινη όραση
Όταν μειώσαμε το delay() στον κώδικα, το LED θα αναβοσβηνε όλο και πιο γρήγορα.

- Αν ορίσουμε delay(10); (δηλαδή 10ms αναμμένο και 10ms σβηστό), η συνολική Περίοδος γίνεται **10+10=20ms**
- Η Συχνότητα τότε γίνεται: $f = \frac{1}{20ms} = \frac{1}{0.02} = 50Hz$
- Δηλαδή το LED αναβοσβήνει 50 φορές το δευτερόλεπτο!

Η «Απάτη» των ματιών μας (Flicker Fusion):

Το ανθρώπινο οπτικό σύστημα έχει ένα συγκεκριμένο βιολογικό όριο ταχύτητας. Όταν ένα φως αναβοσβήνει με περίοδο μικρότερη από **12ms έως 15ms** (κάτι που αντιστοιχεί σε συχνότητες πάνω από **70Hz με 85Hz**), ο εγκέφαλος **δεν προλαβαίνει** να καταγράψει το ενδιάμεσο σβήσιμο.

Το φαινόμενο αυτό ονομάζεται **Flicker Fusion**. Για τον παρατηρητή, το LED σταματά να τρεμοπαίζει και φαίνεται **μόνιμα αναμμένο** (απλά με ελαφρώς χαμηλότερη φωτεινότητα).
Με αυτήν ακριβώς την «απάτη» λειτουργεί και η σύγχρονη τεχνολογία:

- **Οθόνες Κινητών & PC:** Λειτουργούν στα 60Hz, 90Hz ή 120Hz ανανεώνοντας την εικόνα τόσο γρήγορα ώστε να βλέπουμε απόλυτα σταθερά γραφικά και ομαλή κίνηση.
- **Σινεμά:** Οι ταινίες προβάλλουν 24 σταθερές φωτογραφίες (καρέ) το δευτερόλεπτο, αλλά ένας μηχανισμός τις "κόβει" με φως δύο ή τρεις φορές (ανεβάζοντας τη συχνότητα στα 48Hz ή 72Hz), ώστε να βλέπουμε συνεχή ροή χωρίς να κουράζονται τα μάτια μας.

Στο Passive Buzzer, αντί να ξεγελάσουμε το μάτι αναβοσβήνοτνας το LED, **κουνάμε τον αέρα**!Όταν γράφουμε την εντολή **tone(buzzerPin, 440);** (που αντιστοιχεί στη μουσική νότα Λα), το Arduino αναβοσβήνει το Pin 3 με συχνότητα 440 φορών το δευτερόλεπτο. Αυτό σημαίνει ότι η περίοδος της δόνησης είναι μόλις **2.2ms.**
Αυτή η απίστευτα γρήγορη ηλεκτρική εναλλαγή αναγκάζει την εσωτερική μεμβράνη του buzzer να δονηθεί. Η δόνηση μεταφέρεται στον αέρα, παράγονται ηχητικά κύματα, και το αυτί μας τη μεταφράζει σε καθαρό μουσικό ήχο.

Απαντηση

- **Περίοδος:** Ενας **πλήρης** κύκλος (Αναμμένο + Σβηστό) διαρκεί 1000ms + 1000ms = 2000ms) (δηλαδή **2 δευτερόλεπτα**). Άρα **T = 2s**
- **Συχνότητα:** Μέσα σε 1 δευτερόλεπτο προλαβαίνει να γίνει μόνο ο μισός κύκλος. Κάνοντας τη διαίρεση βρίσκουμε ότι η συχνότητα είναι **0.5Hz.**

Προγραμματισμός: Οι συναρτήσεις του ήχου

Για να ελέγξουμε τις νότες, τη συχνότητα και τη διάρκεια του ήχου στο Passive Buzzer Module, το Arduino διαθέτει δύο έτοιμες και πολύ ισχυρές συναρτήσεις:

Α) Η συνάρτηση tone()

Η συνάρτηση αυτή δημιουργεί έναν τετραγωνικό παλμό (συχνότητα) στο pin εξόδου, αναγκάζοντας τη μεμβράνη του buzzer να δονηθεί και να παράξει μια συγκεκριμένη μουσική νότα.

```
tone(pin, frequency);
// ή
tone(pin, frequency, duration);
```

- **pin:** Το ψηφιακό pin του Arduino που στέλνει το σήμα στο module (στο μάθημά μας, το **3**).
- **frequency:** Η συχνότητα του ήχου σε Hertz (Hz). Όσο πιο μικρός είναι ο αριθμός, τόσο πιο μπάσος είναι ο ήχος. Όσο πιο μεγάλος, τόσο πιο οξύς/τσιριχτός (π.χ. το 262 είναι η νότα Ντο, το 440 είναι η νότα Λα).
- **duration (προαιρετικό):** Η διάρκεια του ήχου σε milliseconds (χιλιοστά του δευτερολέπτου).

Β) Η συνάρτηση noTone()

Σταματάει αμέσως την παραγωγή ήχου στο συγκεκριμένο pin. Αν παίξουμε μια νότα με την tone() χωρίς να ορίσουμε διάρκεια, το buzzer θα συνεχίσει να σφυρίζει για πάντα μέχρι να του δώσουμε την εντολή noTone().

```
noTone(pin);
```

Παράδειγμα προγράμματος 1)
Αυτός ο κώδικας αλλάζει συνεχώς τη συχνότητα που στέλνει στο module (στο Pin 3), δημιουργώντας έναν εναλλασσόμενο ήχο (σαν σειρήνα ασθενοφόρου).

```
const int buzzerPin = 3; // Το σήμα του Passive Buzzer στο Pin 3

void setup() {
  pinMode(buzzerPin, OUTPUT); // Ορισμός του Pin 3 ως Έξοδο
}

void loop() {
  tone(buzzerPin, 440); // Παίζει τη συχνότητα 440Hz (Νότα Λα)
  delay(500);           // Κρατάει τον ήχο για μισό δευτερόλεπτο
  tone(buzzerPin, 880); // Παίζει τη συχνότητα 880Hz (Μια οκτάβα πιο ψηλά)
  delay(500);           // Κρατάει τον ήχο για μισό δευτερόλεπτο
}
```

Έλεγχος:

1. Πάτα **Upload** στο Arduino IDE.
2. Βεβαιώσου ότι οι θύρες COM/ttyUSB είναι σωστά επιλεγμένες.
3. **Τι πρέπει να ακούς:** Το buzzer πρέπει να εναλλάσσει αυτόματα τον ήχο του ανά μισό δευτερόλεπτο χωρίς σταματημό

Παράδειγμα προγράμματος 2: Ήχος με Διακοπή (ΜπιπBlink η BlinkΜπιπ)
Αυτός ο κώδικας παράγει ένα σύντομο ηχητικό σήμα (μπιπ) και μετά χρησιμοποιεί τη noTone() για να δημιουργήσει ένα καθαρό κενό σιωπής ενός δευτερολέπτου.

```
const int buzzerPin = 3; // Το σήμα του Passive Buzzer στο Pin 3

void setup() {
  pinMode(buzzerPin, OUTPUT); // Ορισμός του Pin 3 ως Έξοδο
}

void loop() {
  tone(buzzerPin, 1000); // Παίζει έναν καθαρό ήχο στα 1000Hz
  delay(150);            // Αφήνει τον ήχο να ακουστεί για 150ms
  noTone(buzzerPin);     // Σταματάει τον ήχο τελείως
  delay(1000);           // Περιμένει 1 δευτερόλεπτο στη σιωπή πριν το επόμενο μπιπ
}
```

Έλεγχος:

1. Πάτα **Upload** στο Arduino IDE.
2. **Τι πρέπει να ακούς:** Το buzzer πρέπει να κάνει ένα κοφτό "μπιπ" κάθε ένα δευτερόλεπτο, θυμίζοντας τον ήχο που κάνει ένα ξυπνητήρι.

Αποστολή 1

- Τροποποιήστε τον κώδικα ώστε το buzzer να μην κάνει απλά ένα μόνιμο μπιπ, αλλά να εκπέμπει το διεθνές σήμα κινδύνου SOS σε κώδικα Μορς. Το SOS αποτελείται από 3 σύντομα μπιπ, 3 παρατεταμένα μπιπ και 3 σύντομα μπιπ.

Αποστολή 2

- Φτιάξτε το ηχητικό εφέ μιας αντίστροφης μέτρησης! Το buzzer πρέπει να κάνει σύντομα μπιπ τα οποία όμως θα γίνονται όλο και πιο γρήγορα μεταξύ τους (π.χ. το delay ανάμεσα στα μπιπ να ξεκινάει από τα 1000ms και να μειώνεται σταδιακά μέχρι το 0, όπου εκεί θα ακουστεί ένας μεγάλος, συνεχόμενος ήχος "έκρηξης").
  Πρώτα διαβάζουμε την εξήγηση και τα παραδείγματα της εντολής for.

Η εντολή for

Η for επαναλαμβάνει εντολές πολλές φορές. Χρησιμοποιούμε έναν **μετρητή** για να μετράμε πόσες φορές θα εκτελεστεί ο κώδικας.

```
for (int i = 0; i < 5; i++) {
  // εντολές που επαναλαμβάνονται
}
```

Εδώ:

- Το int i = 0 ξεκινά τη μέτρηση από το 0.
- Το i < 5 λέει ότι η επανάληψη θα συνεχίζεται όσο το i είναι μικρότερο από 5.
- Το i++ αυξάνει το i κατά 1 κάθε φορά.

η τιμή του i αλλάζει έτσι:

Επανάληψη	**Τιμή του i**
1η	0
2η	1
3η	2
4η	3
5η	4
6η	5 (βγάινει)

Παράδειγμα:

```
void setup() {
  pinMode(ledPin, OUTPUT);

  for (int delayTime = 100; delayTime >= 1000; delayTime += 100) {
    digitalWrite(ledPin, HIGH);
    delay(delayTime);
    digitalWrite(ledPin, LOW);
    delay(delayTime);
  }
}
```

Σε αυτό το παράδειγμα αλλάζουμε τον χρόνο του delay κατά 200. Ξεκινάμε με delay 100ms και σε κάθε επανάληψη αυξάνεται κατά 100 μέχρι να φτάσει τα 1000ms.
Αν γράφαμε:

```
  for (int delayTime = 1000; delayTime <= 100; delayTime -= 100) {
```

θα είχαμε το αντίθετο. Το delay αρχίζει με τιμή 1000ms και σε κάθε επανάληψη μειώνεται κατά 100 μέχρι να φτάσει το 0.

Αποστολή 3: "Theremin - Μουσική με Σκιές" (Boss Level)
Στόχος: Συνδυάστε το LDR Module (Pin A0) από το προηγούμενο μάθημα μαζί με το Buzzer Module (Pin 3).
Φτιάξτε ένα φουτουριστικό μουσικό όργανο που αλλάζει ήχο χωρίς να το ακουμπάτε! Όσο πλησιάζετε το χέρι σας στο LDR και του κλείνετε το φως, η συχνότητα του ήχου στο buzzer πρέπει να αλλάζει αυτόματα (να γίνεται πιο μπάσα ή πιο πρίμα).
• Hint: Χρησιμοποιήστε τη συνάρτηση analogRead(A0) και τη συνάρτηση **map()** για να μετατρέψετε απευθείας τις τιμές του φωτός (0-1023) σε ακουστικές συχνότητες (π.χ. από 150 έως 1500Hz) μέσα στην tone().

Η συνάρτηση map()

Η συνάρτηση map() μετατρέπει έναν αριθμό (**value**) από ένα εύρος τιμών σε ένα άλλο. Δηλαδή:

1. η τιμή **fromLow** αντιστοιχίζεται στην τιμή **toLow**,
2. η τιμή **fromHigh** στην τιμή **toHigh**
3. και οι ενδιάμεσες τιμές αντιστοιχίζονται αναλογικά.

```
map(value, fromLow, fromHigh, toLow, toHigh)
```

Παράδειγμα

```
void loop() {
  int val = analogRead(0);

  Serial.print("Input value: ");
  Serial.print(val);

  val = map(val, 0, 1023, 0, 255); // mapping function

  Serial.print(", Output value: ");
  Serial.println(val);
```

Η μεταβλητή **val** θα πάρει μια τιμή από 0 έως 1023.
Στη συνέχεια αυτές οι τιμές θα αντιστοιχούν στα νέα όρια, δηλαδή 0 έως 255.
Άσκηση
Συμπληρώστε τα κενά, γράφοντας δίπλα σε κάθε παράμετρο της map() την αντίστοιχη τιμή της από το παράδειγμα.

```
val = map(val, 0, 1023, 0, 255);
```

- toHigh = _____
- value = _____
- fromHigh = _____
- fromLow = _____
- toLow = _____

Μάθημα 4.5 –Making Music

Αποστολή 1: "Φτιάξε ένα Πιάνο!"

Στόχος: Σύνδεσε 3 Button modules (για τις νότες Ντο, Ρε, Μι) και το Passive Buzzer module. Όταν πατάς ένα κουμπί, το buzzer πρέπει να παίζει την αντίστοιχη συχνότητα από τον πίνακα (C=262Hz, D=294Hz, E=330Hz). Όταν το αφήνεις, ο ήχος πρέπει να σταματάει.

```
#define NOTE_C 262
#define NOTE_D 294
#define NOTE_E 330
..
..
..
const int buzzerPin = 3;
const int btnC = 4;
const int btnD = 6;
const int btnE = 7;

void setup() {
  pinMode(buzzerPin, OUTPUT);
  pinMode(btnC, INPUT);
  pinMode(btnD, INPUT);
  pinMode(btnE, INPUT);
}

void loop() {
  // Έλεγχος αν πατήθηκε το πρώτο κουμπί (Ντο)
  if (digitalRead(btnC) == HIGH) {
    tone(buzzerPin, NOTE_C);
  }
...
  else {
    noTone(buzzerPin);
  }
}
```

NOTE FREQUENCY CHART | HEROIC AUDIO

	Octave 0	Octave 1	Octave 2	Octave 3	Octave 4	Octave 5	Octave 6	Octave 7	Octave 8	Octave 9	Octave 10
C	16.35	32.70	65.41	130.81	261.63	523.25	1046.50	2093.00	4186.01	8372.02	16744.04
C#	17.32	34.65	69.30	138.59	277.18	554.37	1108.73	2217.46	4434.92	8869.84	17739.69
D	18.35	36.71	73.42	146.83	293.66	587.33	1174.66	2349.32	4698.64	9397.27	18794.55
D#	19.45	38.89	77.78	155.56	311.13	622.25	1244.51	2489.02	4978.03	9956.06	19912.13
E	20.60	41.20	82.41	164.81	329.63	659.26	1318.51	2637.02	5274.04	10548.08	
F	21.83	43.65	87.31	174.61	349.23	698.46	1396.91	2793.83	5587.65	11175.30	
F#	23.12	46.25	92.50	185.00	369.99	739.99	1479.98	2959.96	5919.91	11839.82	
G	24.50	49.00	98.00	196.00	392.00	783.99	1567.98	3135.96	6271.93	12543.86	
G#	25.96	51.91	103.83	207.65	415.30	830.61	1661.22	3322.44	6644.88	13289.75	
A	27.50	55.00	110.00	220.00	440.00	880.00	1760.00	3520.00	7040.00	14080.00	
A#	29.14	58.27	116.54	233.08	466.16	932.33	1864.66	3729.31	7458.62	14917.24	
B	30.87	61.74	123.47	246.94	493.88	987.77	1975.53	3951.07	7902.13	15804.26	

Αποστολή 2: «Φτιάξε ένα Synthesizer!»

Στόχος: Χρησιμοποίησε επαναλήψεις for για να δημιουργήσεις διαφορετικούς ήχους. Συμπλήρωσε τον σκελετό του προγράμματος, ώστε κάθε κουμπί να ενεργοποιεί το δικό του ηχητικό εφέ.

- Κουμπί 1 (Pin 4) — Εφέ Laser: Η συχνότητα ξεκινά από τα 1000 Hz και πέφτει σταδιακά μέχρι τα 200 Hz.
- Κουμπί 2 (Pin 5) — Εφέ Σειρήνας: Η συχνότητα ξεκινά από τα 500 Hz, ανεβαίνει μέχρι τα 1200 Hz και έπειτα κατεβαίνει ξανά στα 500 Hz.
- Κουμπί 3 (Pin 6) — Εφέ Ανόδου: Η συχνότητα ξεκινά από τα 200 Hz και ανεβαίνει μέχρι τα 1000 Hz.

Σε κάθε επανάληψη, η συχνότητα αλλάζει κατά 10 Hz. Χρησιμοποίησε delay(5) για να ακούγεται κάθε συχνότητα για λίγο.

```
const int buzzerPin = 3;
const int btnLaser = 4;
const int btnSiren = 5;
const int btnRise = 6;

void setup() {
  pinMode(buzzerPin, OUTPUT);
  pinMode(btnLaser, INPUT);
  pinMode(btnSiren, INPUT);
  pinMode(btnRise, INPUT);
}

void loop() {
  // Κουμπί 1: Εφέ Laser
  if (digitalRead(btnLaser) == HIGH) {
    // Συμπλήρωσε μια for:
    // Ξεκινά από τα 1000 Hz και κατεβαίνει μέχρι τα 200 Hz.
    // Σε κάθε επανάληψη:
    // 1. Κάλεσε tone(buzzerPin, freq)
    // 2. Περίμενε 5 ms με delay(5)
    noTone(buzzerPin);
  }
  // Κουμπί 2: Εφέ Σειρήνας
  else if (digitalRead(btnSiren) == HIGH) {
    // Συμπλήρωσε μία for που ανεβάζει τη συχνότητα
    // από τα 500 Hz μέχρι τα 1200 Hz.

    // Συμπλήρωσε μία for που κατεβάζει τη συχνότητα
    // από τα 1200 Hz μέχρι τα 500 Hz.
    noTone(buzzerPin);
  }
  // Κουμπί 3: Εφέ Ανόδου
  else if (digitalRead(btnRise) == HIGH) {
    // Συμπλήρωσε μια for:
    // Ξεκινά από τα 200 Hz και ανεβαίνει μέχρι τα 1000 Hz.
    noTone(buzzerPin);
  }
}
```

Αποστολη 3

Πειραματιστείτε με τα περιεχόμενα του παρακάτω repo.

https://github.com/robsoncouto/arduino-songs

Theremin Solution

```
const int ldrPin = A0;     // Το LDR Module στην Αναλογική θύρα A0
const int buzzerPin = 3;   // Το Passive Buzzer Module στο Pin 3

int ldrValue = 0;          // Μεταβλητή για την αποθήκευση της τιμής του φωτός
int frequency = 0;         // Μεταβλητή για τη συχνότητα του ήχου

void setup() {
  pinMode(buzzerPin, OUTPUT); // Ορισμός του Pin 3 ως Έξοδο
  Serial.begin(9600);         // Ενεργοποίηση Serial Monitor για έλεγχο
}

void loop() {
  // 1. Διαβάζουμε την αναλογική τιμή του φωτός (από 0 έως 1023)
  ldrValue = analogRead(ldrPin);

  // 2. Η ΣΥΝΑΡΤΗΣΗ-ΚΛΕΙΔΙ: Μετατρέπουμε το εύρος του LDR (0-1023)
  // σε ένα ακουστικό εύρος συχνοτήτων για το buzzer (π.χ. 150Hz έως 1500Hz)
  frequency = map(ldrValue, 0, 1023, 150, 1500);

  // 3. Στέλνουμε τη συχνότητα στο buzzer για να παραχθεί ο ήχος
  tone(buzzerPin, frequency);

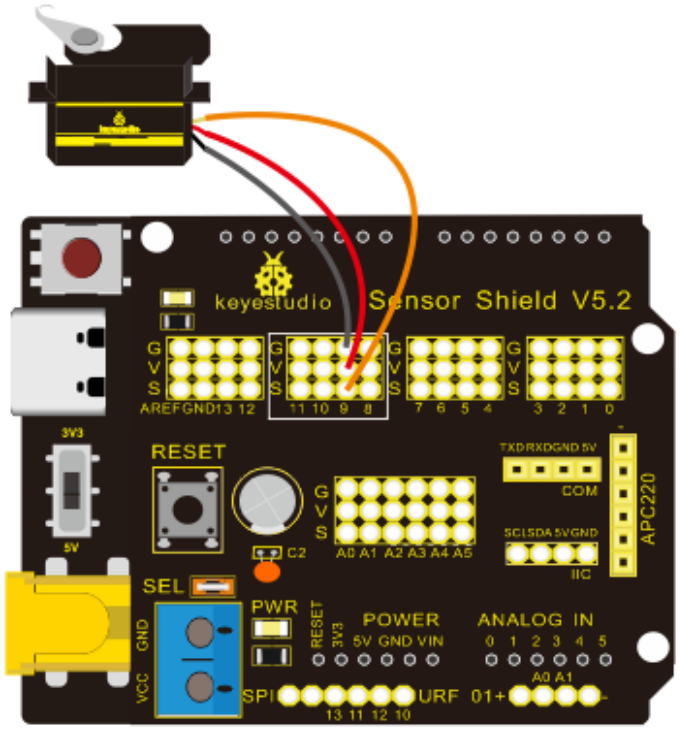
  // Εκτύπωση των τιμών στον υπολογιστή για έλεγχο και κατανόηση
  Serial.print("Φως (LDR): ");
  Serial.print(ldrValue);
  Serial.print("  ->  Συχνότητα (Hz): ");
  Serial.println(frequency);

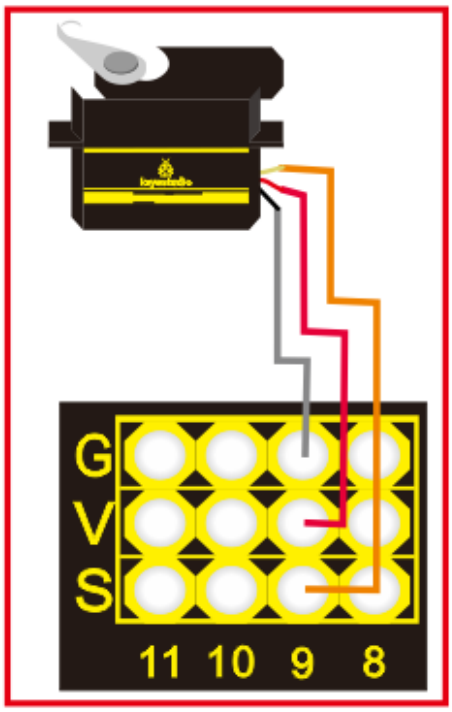
  delay(10); // Πολύ μικρή καθυστέρηση για άμεση και ομαλή ανταπόκριση στον ήχο
}
```

Μάθημα 05 – Servo Motors

Συνδεσμολογία

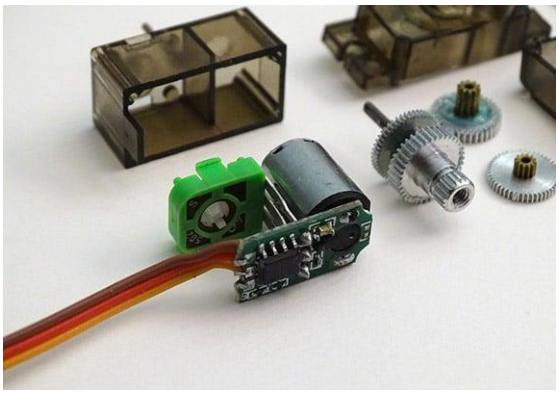
Σύνδεσε το Servo motor odule στα G, V και D9.

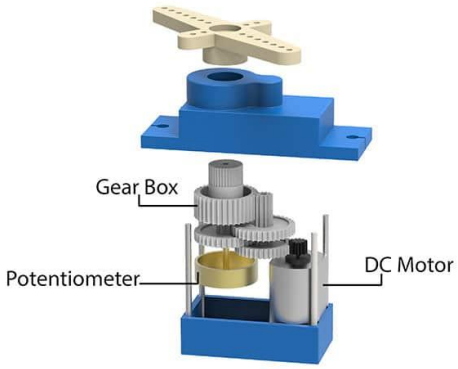




Ένας κινητήρας servo μπορεί να χρησιμοποιηθεί για να κουνήσουμε κάτι. Στο smart Home μας θα ελέγχει πόρτες και παράθυρα. Σε αυτό το μάθημα θα παρουσιάσουμε τη βασική αρχή λειτουργίας του και θα δείξουμε πώς χρησιμοποιείται.

Το servo είναι ένας περιστροφικός μηχανισμός **που ελέγχει τη θέση του**. Αποτελείται κυρίως από το περίβλημα, ένα ηλεκτρονικό κύκλωμα, έναν ηλεκτρικό κινητήρα, γρανάζια και έναν αισθητήρα θέσης.

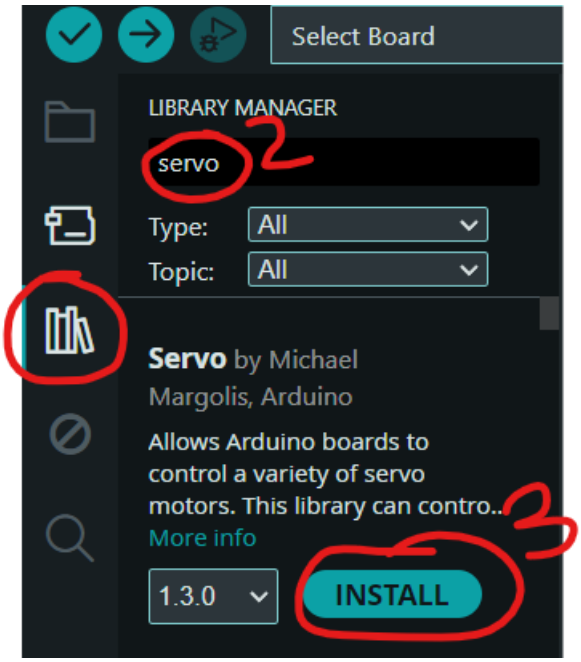




Το servo λαμβάνει ένα σήμα από τον μικροελεγκτή (Arduino). Με βάση αυτό, ο εσωτερικός μηχανισμός συγκρίνει τη θέση που έχει οριστεί με τη θέση που μετρά ο αισθητήρας. Ο κινητήρας κινείται μέχρι να φτάσει στην επιθυμητή θέση και τότε σταματά.
Συνήθως, το servo μπορεί να περιστραφεί από **0° έως 180°**.

Προγραμματισμός

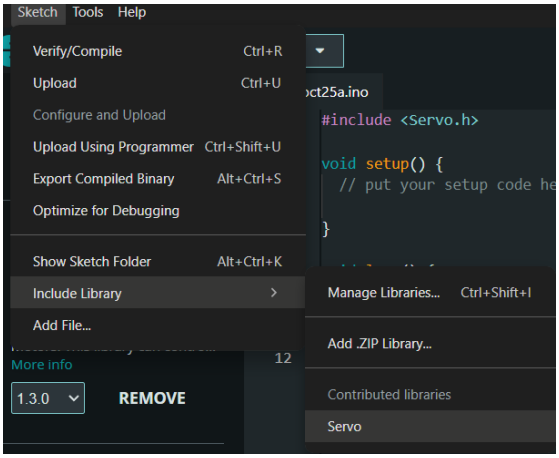
Το servo είναι ένας κινητήρας του οποίου μπορούμε να ορίσουμε τη θέση, δηλαδή τη γωνία περιστροφής. Για να το ελέγξουμε, χρησιμοποιούμε τη βιβλιοθήκη **Servo.h** την οποία πρέπει να εγκαταστήσουμε όπως φαίνεται δεξιά! →



Στην συνέχεια θα πρέπει να την συμπεριλάβουμε στο πρόγραμμα είτε γράφοντας στην αρχή του προγράμματος

```
#include <Servo.h>
```

Είτε πατώντας στο Arduino IDE:
Sketch → Inlude Library → Servo



Για να αλλάξουμε την γωνία του κινητήρα χρησιμοποιούμε την εντολή

```
myservo.write(γωνία);
```

αλλά όπως με το led πρέπει πρώτα να δηλώσουμε pinMode έτσι με και με το Servo πρέπει στην αρχή να δηλώσουμε τον κινητήρα σαν μεταβλητή γράφοντας **Servo myservo**
και στη συνέχεια να πούμε στο Arduino σε ποιο pin είναι γράφοντας **myservo.attach(PIN)**

```
#include <Servo.h>
int pos = 90;
Servo myservo;

void setup() {
  myservo.attach(9);
  myservo.write(pos);

}
```

Παράδειγμα προγράμματος

Το πρόγραμμα κινεί το servo από τις 0° στις 180° και μετά πίσω στις 0°.

```
#include <Servo.h>

Servo myservo;
int pos = 0;

void setup() {
  myservo.attach(9);
}

void loop() {
  // Κίνηση από τις 0° στις 180°
  for (pos = 0; pos <= 180; pos++) {
    myservo.write(pos);
    delay(15);
  }
  // Κίνηση από τις 180° στις 0°
  for (pos = 180; pos >= 0; pos--) {
    myservo.write(pos);
    delay(15);
  }
}
```

Έλεγχος

Πάτα **Verify** και μετά **Upload**. Το servo πρέπει να κινείται ομαλά από τη μία πλευρά στην άλλη και πίσω.
Αν δεν λειτουργεί, έλεγξε:

- Ότι ο κώδικας ανέβηκε χωρίς σφάλμα.
- Ότι το servo είναι συνδεδεμένο στα σωστά pins: **G, V και D9**.
- Ότι το καλώδιο σήματος είναι συνδεδεμένο στο pin που αναφέρεται στο myservo.attach(9)

Η σειριακή οθόνη σαν είσοδος

Η **Serial Monitor** επιτρέπει να στέλνουμε πληροφορίες από τον υπολογιστή στο Arduino. Ως τώρα έχουμε μόνο λάβει πληροφορίες από το Arduino. Ήρθε η στιγμή να στείλουμε και κάτι πίσω! Στο παράδειγμα, γράφουμε έναν αριθμό και το Arduino τον αποθηκεύει και τον εμφανίζει πισω!

```
int number;

void setup() {
  Serial.begin(9600);  // Ξεκινά η σειριακή επικοινωνία
}

void loop() {
  if (Serial.available() > 0) {  // Ελέγχει αν στάλθηκε κάτι
    number = Serial.parseInt();  // Διαβάζει τον αριθμό

    Serial.print("The number is: ");
    Serial.println(number);      // Εμφανίζει τον αριθμό
  }
}
```

Αποστολή: Μάντεψε τον αριθμό!

Το Arduino διαλέγει έναν τυχαίο μυστικό αριθμό από το **1 έως το 100**. Προσπάθησε να τον μαντέψεις γράφοντας αριθμούς στη **Serial Monitor**. Έχεις **10 προσπάθειες**!
Μετά από κάθε αριθμό, το Arduino θα σου λέει αν ο μυστικός αριθμός είναι **μεγαλύτερος** ή **μικρότερος**. Αν τον βρεις, θα εμφανίζει μήνυμα επιτυχίας.
Πρόκληση: Μπορείς να τον βρεις σε **7 προσπάθειες ή λιγότερες**; Χρησιμοποίησε τη δυαδική αναζήτηση: κάθε φορά δοκίμαζε περίπου τον μεσαίο αριθμό του εύρους που απομένει.

Αντέγραψε και διόρθωσε το παρακάτω πρόγραμμα!

```
int secretNumber;
int guess;
int attempts = 0;

void setup() {
  Serial.begin(9600);
  randomSeed(analogRead(A0));
  secretNumber = random(1, 101);
  Serial.println("=== GUESS THE NUMBER ===");
  Serial.println("I have chosen a number from 1 to 100.");
  Serial.println("You have 10 attempts.");
  Serial.println();
}
void loop() {
  Serial.println("------------------------");
  Serial.print("Attempts left: ");
  Serial.println(10 - attempts);
  Serial.print("Enter your guess: ");
  while (Serial.available() == 0) {}// Περιμένει να γράψεις έναν αριθμό
  guess = Serial.parseInt();
  // Καθαρίζει το Enter και τυχόν υπόλοιπους χαρακτήρες
  while (Serial.available() > 0) {Serial.read();}

  attempts++;
  Serial.print("Your guess: ");
  Serial.println(guess);

  if (ελεγχος αν ο αριθμος ειναι ο ιδιος) {
    Serial.println("Correct! You found the number!");
    Serial.println("A new game is starting.");
    attempts = 0;
    secretNumber = random(1, 101);
  }
  else if (ελεγχος αν ο αριθμος ειναι μικροτερος) {
    Serial.println("Try a bigger number.");
  }
  else {
    Serial.println("Try a smaller number.");
  }
  if (ελεγχος αν ο αριθμος ειναι 10 και ο αριθμος δεν ειναι ο ιδιος) {
    Serial.println();
    Serial.println("No attempts left!");
    Serial.print("The number was: ");
    Serial.println(secretNumber);
    Serial.println("A new game is starting.");
    attempts = 0;
    secretNumber = random(1, 101);
  }
  Serial.println();
```

Αποστολή 2

Φτιάξτε ένα πρόγραμμα στο οποίο θα δίνουμε μέσω σειριακής οθόνης έναν αριθμό μεταξύ 0 και 180 και αυτός ο αριθμός γίνεται περιστροφή του κινητήρα servo.

Serial monitor input Servo Solution

```
#include <Servo.h>

Servo myservo;
int angle;

void setup() {
  Serial.begin(9600);
  myservo.attach(9);

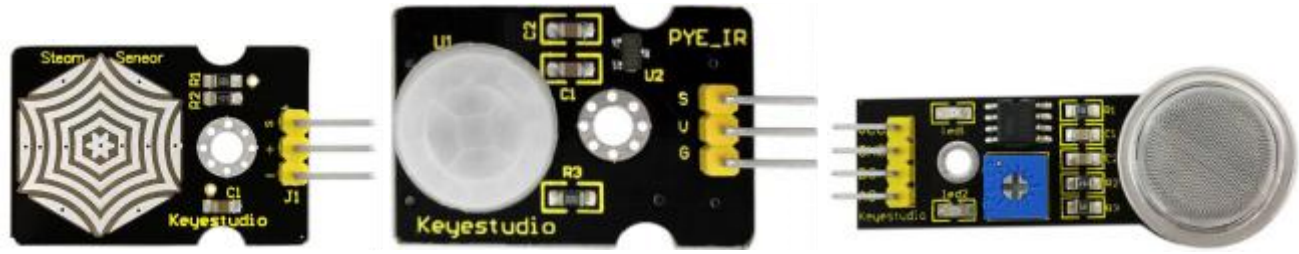
  Serial.println("Enter an angle from 0 to 180:");
}

void loop() {
  if (Serial.available() > 0) {
    angle = Serial.parseInt();

    if (angle >= 0 && angle <= 180) {
      myservo.write(angle);
      Serial.print("Servo angle: ");
      Serial.println(angle);
    } else {
      Serial.println("Enter a number from 0 to 180.");
    }

    while (Serial.available() > 0) {
      Serial.read();  // Καθαρίζει το Enter
    }
  }
}
```

Μάθημα 06 – Other Sensors (Steam, PIR, Gas)



Steam Sensor (1)

Πρόκειται για αισθητήρα που ανιχνεύει την παρουσία νερού στην επιφάνειά του. Διαθέτει παράλληλες, εκτεθειμένες αγώγιμες γραμμές. Όταν το νερό τις καλύπτει, η ηλεκτρική αγωγιμότητα μεταξύ τους αλλάζει και μεταβάλλεται η αναλογική έξοδος του αισθητήρα. Μπορεί να χρησιμοποιηθεί ως ανιχνευτής βροχής ή για να εντοπίζει νερό σε μια επιφάνεια.
Προσοχή: Το σημείο σύνδεσης του αισθητήρα δεν είναι αδιάβροχο. Μην βυθίζετε τον αισθητήρα στο νερό.

PIR Motion Sensor(2)

Ο αισθητήρας υπέρυθρης κίνησης PIR ανιχνεύει αλλαγές στην υπέρυθρη ακτινοβολία του περιβάλλοντος, όπως αυτές που προκαλούνται όταν κινείται ένα ανθρώπινο σώμα μπροστά του. Όταν ανιχνεύσει κίνηση, η ψηφιακή έξοδός του αλλάζει κατάσταση.

Gas Sensor (3)

Ο αισθητήρας αερίων MQ-2 χρησιμοποιείται σε συστήματα ανίχνευσης εύφλεκτων αερίων και καπνού. Ανταποκρίνεται σε αέρια όπως το υγραέριο, το φυσικό αέριο, το υδρογόνο, η αλκοόλη και το βενζόλιο, καθώς και στον καπνό.Ο αισθητήρας χρειάζεται χρόνο προθέρμανσης πριν από τη χρήση. Η αναλογική του έξοδος μεταβάλλεται ανάλογα με την παρουσία και τη συγκέντρωση αερίων ή καπνού. Η ευαισθησία του module μπορεί να ρυθμιστεί περιστρέφοντας το ποτενσιόμετρο.

Πως θα τον συνδέσετε κάθε αισθητήρα και ποιες εντολές θα χρησιμοποιήσετε για να διαβάσετε τις τιμές τους;

……

Αποστολή!

Φτιάξτε ένα ολοκληρωμένο σύστημα ασφαλείας για το σπίτι. Αν ανιχνευθεί βροχή **ή** εισβολέας **ή** κάποιο επικίνδυνο αέριο θα πρέπει να σημάνει συναγερμός!

Οδηγίες Κατασκευής – Προϋπόθεση να έχετε ολοκληρώσει το Serial Monitor Servo Input!

https://docs.keyestudio.com/projects/KS0085/en/latest/docs/2.%20Product%20installation/2.%20Product%20installation.html